```
        if (t < sqrt(FPMIN)) {                    Special case: Avoid failure of convergence
            sumc=0.0;                                 test because of underflow.
            sums=t;
        } else {
            sum=sums=sumc=0.0;
            sign=fact=1.0;
            odd=true;
            for (k=1;k<=MAXIT;k++) {
                fact *= t/k;
                term=fact/k;
                sum += sign*term;
                err=term/abs(sum);
                if (odd) {
                    sign = -sign;
                    sums=sum;
                    sum=sumc;
                } else {
                    sumc=sum;
                    sum=sums;
                }
                if (err < EPS) break;
                odd=!odd;
            }
            if (k > MAXIT) throw("maxits exceeded in cisi");
        }
        cs=Complex(sumc+log(t)+EULER,sums);
    }
    if (x < 0.0) cs = conj(cs);
    return cs;
}
```

CITED REFERENCES AND FURTHER READING:

Stegun, I.A., and Zucker, R. 1976, "Automatic Computing Methods for Special Functions. III. The Sine, Cosine, Exponential integrals, and Related Functions," *Journal of Research of the National Bureau of Standards*, vol. 80B, pp. 291–311; 1981, "Automatic Computing Methods for Special Functions. IV. Complex Error Function, Fresnel Integrals, and Other Related Functions," *op. cit.*, vol. 86, pp. 661–686.

Abramowitz, M., and Stegun, I.A. 1964, *Handbook of Mathematical Functions* (Washington: National Bureau of Standards); reprinted 1968 (New York: Dover); online at `http://www.nr.com/aands`, Chapters 5 and 7.

6.9 Dawson's Integral

Dawson's Integral $F(x)$ is defined by

$$F(x) = e^{-x^2} \int_0^x e^{t^2}\, dt \tag{6.9.1}$$

See Figure 6.8.1 for a graph of the function. The function can also be related to the complex error function by

$$F(z) = \frac{i\sqrt{\pi}}{2} e^{-z^2} \left[1 - \mathrm{erfc}(-iz)\right]. \tag{6.9.2}$$

A remarkable approximation for $F(z)$, due to Rybicki [1], is

$$F(z) = \lim_{h \to 0} \frac{1}{\sqrt{\pi}} \sum_{n \text{ odd}} \frac{e^{-(z-nh)^2}}{n} \tag{6.9.3}$$

What makes equation (6.9.3) unusual is that its accuracy increases *exponentially* as h gets small, so that quite moderate values of h (and correspondingly quite rapid convergence of the series) give very accurate approximations.

We will discuss the theory that leads to equation (6.9.3) later, in §13.11, as an interesting application of Fourier methods. Here we simply implement a routine for real values of x based on the formula.

It is first convenient to shift the summation index to center it approximately on the maximum of the exponential term. Define n_0 to be the even integer nearest to x/h, and $x_0 \equiv n_0 h$, $x' \equiv x - x_0$, and $n' \equiv n - n_0$, so that

$$F(x) \approx \frac{1}{\sqrt{\pi}} \sum_{n'=-N \text{ odd}}^{N} \frac{e^{-(x'-n'h)^2}}{n' + n_0} \tag{6.9.4}$$

where the approximate equality is accurate when h is sufficiently small and N is sufficiently large. The computation of this formula can be greatly speeded up if we note that

$$e^{-(x'-n'h)^2} = e^{-x'^2} e^{-(n'h)^2} \left(e^{2x'h}\right)^{n'} \tag{6.9.5}$$

The first factor is computed once, the second is an array of constants to be stored, and the third can be computed recursively, so that only two exponentials need be evaluated. Advantage is also taken of the symmetry of the coefficients $e^{-(n'h)^2}$ by breaking up the summation into positive and negative values of n' separately.

In the following routine, the choices $h = 0.4$ and $N = 11$ are made. Because of the symmetry of the summations and the restriction to odd values of n, the limits on the `for` loops are 0 to 5. The accuracy of the result in this version is about 2×10^{-7}. In order to maintain relative accuracy near $x = 0$, where $F(x)$ vanishes, the program branches to the evaluation of the power series [2] for $F(x)$, for $|x| < 0.2$.

dawson.h

```
Doub dawson(const Doub x) {
Returns Dawson's integral F(x) = exp(−x²) ∫₀ˣ exp(t²)dt for any real x.
    static const Int NMAX=6;
    static VecDoub c(NMAX);
    static Bool init = true;
    static const Doub H=0.4, A1=2.0/3.0, A2=0.4, A3=2.0/7.0;
    Int i,n0;                          Flag is true if we need to initialize, else false.
    Doub d1,d2,e1,e2,sum,x2,xp,xx,ans;
    if (init) {
        init=false;
        for (i=0;i<NMAX;i++) c[i]=exp(-SQR((2.0*i+1.0)*H));
    }
    if (abs(x) < 0.2) {                Use series expansion.
        x2=x*x;
        ans=x*(1.0-A1*x2*(1.0-A2*x2*(1.0-A3*x2)));
    } else {                           Use sampling theorem representation.
        xx=abs(x);
        n0=2*Int(0.5*xx/H+0.5);
        xp=xx-n0*H;
        e1=exp(2.0*xp*H);
```